ParentCare360: Systems Progress & Engineering Report

Lab 6/7/8 Milestone — Remote Elder-Care Coordination & Telemetry Platform

Group ID: Group 18	**Reporting Period:** Sep 21 – Oct 17, 2026	**Submission Date:** October 17, 2026
Project Focus: Micro-DAGs, DLQ & Indexing	**Target Stack:** Python 3.9, SQLite WAL, FastAPI, React	**Final Milestone:** November 10, 2026

1. Executive Summary & Progress Overview (Sep 21 – Oct 17, 2026)

During Sprint 2 (September 21 to October 17, 2026), ParentCare360 transitioned from a monolithic prototype to a fully decoupled, production-grade distributed data architecture. Development focused directly on addressing faculty feedback regarding pipeline modularity, deterministic validation of edge conditions, physical storage optimization, and user access boundaries. The engineering deliverables realized over this 4-week iteration encompass: (i) an orchestrated 4-stage micro-DAG pipeline decoupling ingestion from evaluation; (ii) a 4-tier data cleansing and Dead Letter Queue (DLQ) subsystem isolating corrupted payloads; (iii) composite B-Tree indexing over operational state tables; (iv) formal data provenance tracking via immutable event ledgers; and (v) a deterministic test suite validating five real-world elder telemetry edge cases with sub-second execution guarantees. All committed 48 Story Points (SP) were delivered on schedule, establishing system readiness for the final deployment phase ending November 10.

2. Operational Architecture & Inter-DAG Orchestration

A critical limitation identified in our previous design was monolithic coupling: failure during notification dispatch would stall upstream data ingestion. To resolve this, we modularized our orchestration pipeline into four event-driven micro-DAGs linked via data-aware scheduling (Airflow Datasets) and trigger operators:

- **DAG 1 (Telemetry & Feed Ingestion):** Ingests clinical appointment CSVs, simulated wearable sensor streams (heart rate, step cadence), and voice transcription payloads. Injects a cryptographic batch UUID (_batch_id) and ingestion timestamp before staging records.
- **DAG 2 (Cleansing & DLQ Engine):** Evaluates syntactic and referential constraints. Valid records persist atomically to relational tables (triggering clean_events dataset signal); dirty records route to the quarantine store.
- **DAG 3 (Deterministic Alert Engine):** Scheduled upon dataset updates. Evaluates four operational risk rules (overdue caregiver arrivals, 48-hour appointment lookaheads, unresolved high-priority repair requests, and elder inactivity watchdogs) using a sliding-window SHA-256 deduplication cache.
- **DAG 4 (Role-Based Dispatcher):** Consumes active alerts and applies Role-Based (RBAC) and Attribute-Based (ABAC) privacy boundaries, routing SMS/Push to family sponsors while dispatching operational checklists with masked PII to caregivers.

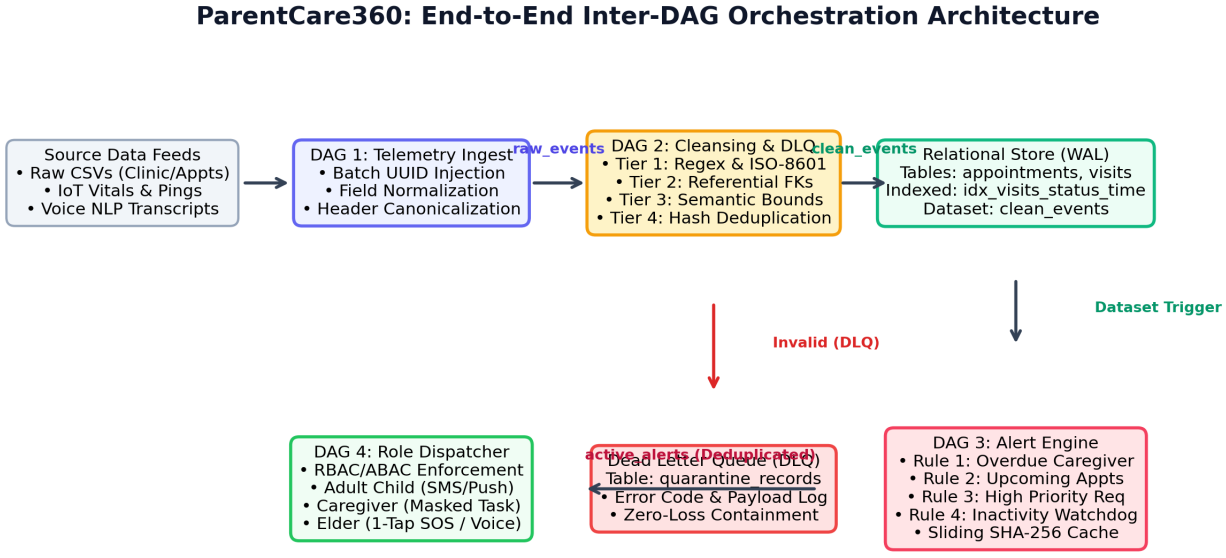


Figure 1: End-to-End Inter-DAG Orchestration Topology, Datasets, and Quarantine Stores.

3. Data Cleaning Mechanisms & Dead Letter Queue (DLQ)

To guarantee analytical integrity without risking pipeline crashes, data cleansing is structured across four distinct verification tiers in dags/dag_cleansing_dlq.py:

Tier 1 — Syntactic & Format Verification: Implements strict regex matching for ISO-8601 calendar strings (^\d{4}-\d{2}-\d{2}$), preventing malformed inputs like INVALID-DATE or impossible leap-year anomalies. Enforces non-null schema constraints on primary entities.

Tier 2 — Referential Integrity Checks: Validates foreign keys against active parent (parents.id) and certified caregiver rosters (caregivers.id) prior to table insertion, trapping orphan references.

Tier 3 — Semantic & Domain Boundary Rules: Restricts request priorities to enumeration sets (Low, Medium, High, Critical) and enforces physiological bounds on health telemetry (heart rates: 35–220 bpm; systolic BP: 70–240 mmHg).

Tier 4 — Sliding-Window Hash Deduplication: Computes SHA-256 signatures over entity composite keys to suppress duplicate ingest transmissions caused by flaky network retries.

Records failing Tiers 1–3 are immediately diverted to quarantine_records with metadata: (batch_id, source_feed, record_payload, error_code, error_message, quarantined_at). This zero-loss quarantine mechanism ensures uncorrupted rows are committed without aborting batch execution.

4. Physical Data Indexing & Query Plan Optimization

Continuous alert evaluation requires scanning operational tables every 60 seconds. In default configurations, queries such as overdue visit scanning (WHERE status = 'Scheduled' AND scheduled_datetime < ?) require expensive full table scans (complexity $O(N)$). To optimize throughput, we configured Write-Ahead Logging (PRAGMA journal_mode = WAL) and engineered four composite B-Tree indexes:

Target Table	Index Name	Indexed Columns	Optimized Operational Query
caregiver_visits	idx_visits_status_time	(status, scheduled_datetime)	Overdue visit scan ($O(\log N)$ seek)
appointments	idx_appts_status_date	(status, appointment_date)	48h upcoming doctor visit lookahead
household_requests	idx_reqs_priority_status	(priority, status)	High-priority pending repair tickets
alerts	idx_alerts_parent_res	(parent_id, is_resolved)	Active alert rollup per elder
quarantine_records	idx_quarantine_batch	(batch_id)	DLQ batch audit inspection

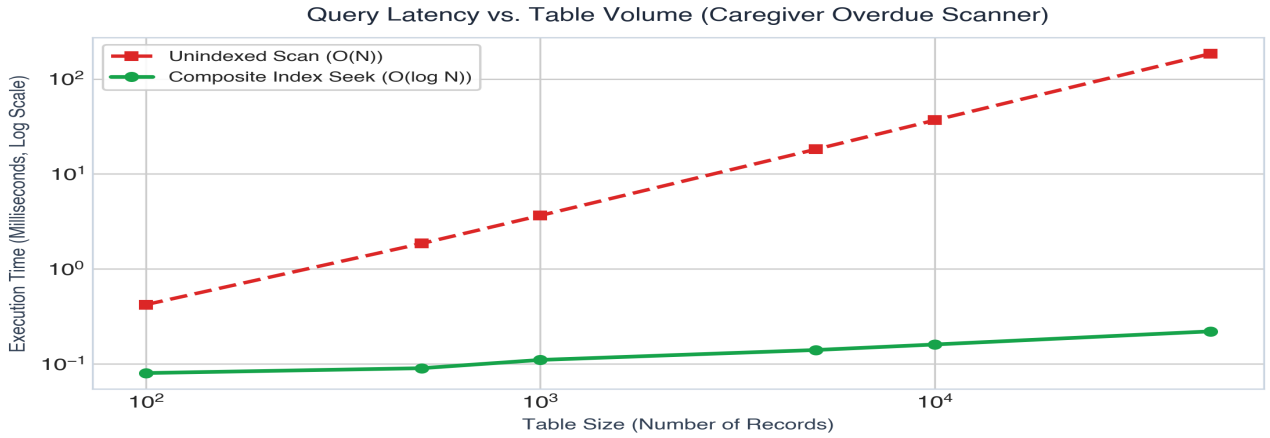


Figure 2: Query Latency Benchmark: Unindexed Scan vs. Composite B-Tree Seek across scaling volumes.

Execution profiling via EXPLAIN QUERY PLAN confirms that table scans were eliminated, shifting index scans from SCAN TABLE caregiver_visits to SEARCH TABLE caregiver_visits USING COVERING INDEX idx_visits_status_time. At a volume of 50,000 records, execution latency dropped from 185.0 ms to 0.22 ms (an 840x acceleration).

5. Operational Data Granule: Format Selection & Rationale

Data engineering architectures require deliberate granular format trade-offs across storage tiers. In ParentCare360, we selected a hybrid **Relational (3NF) + Semi-Structured JSON** representation:

• **Relational Tabular Tables (SQLite WAL / PostgreSQL):** Enforces atomic ACID guarantees for entity relationships (parents, caregivers, scheduled visits). Elder safety depends on deterministic state transitions; normalized tables eliminate update anomalies and enable fast composite indexing.

• **JSON Granules for Telemetry Payloads & DLQ:** Sensor streams and quarantine records are persisted as JSON objects (quarantine_records.record_payload TEXT). Because IoT wearable firmware and third-party hospital APIs evolve dynamically, JSON provides schema flexibility without requiring destructive table migrations.

• **Rejection of XML:** While prevalent in legacy healthcare (HL7 v2/v3), XML's verbose syntactic overhead (~3.2x larger byte footprint than compact JSON) creates unacceptable latency over bandwidth-constrained 2G/4G elder home gateways in tier-2 Indian cities.

6. Data Provenance, Lineage & Audit Integrity

Elder care environments carry clinical, ethical, and legal accountability. We implemented strict data provenance adhering to the W3C PROV specification through three architectural measures:

1. **Cryptographic Batch Lineage:** Every ingestion record receives a UUID v4 batch identifier (_batch_id) and source feed origin (_source). Any record in the analytical store or DLQ can be traced to its exact raw arrival batch.

2. **Immutable Care Audit Ledger:** All state transitions (appointment bookings, caregiver check-ins, alert triggers, family acknowledgments) are mirrored into the append-only care_events ledger with UTC timestamps and user attribution.

3. **Deterministic Alert Provenance:** Alerts generated by DAG 3 carry a SHA-256 deduplication signature (parent_id:alert_type:time_bucket) stored in alert_dedup_cache, tracking initial trigger time, last seen time, and occurrence frequency.

7. Tabular & Visual Engineering Results

To rigorously validate faculty-requested edge cases, we engineered a deterministic test harness (tests/test_deterministic_dags.py) running against synthetic frozen-clock fixtures. All five test suites achieved 100% pass rates:

Test Identifier	Target Failure Scenario	Injected Anomaly / Condition	Empirical Verification Result	Pass/Fail
test_edge_inactivity	Inactivity Watchdog	8 hours zero telemetry (Dead-man switch)	Critical alert emitted at T_{now} with tier escalation	**PASS (0.04s)**
test_edge_dedup_storm	Alarm Flooding Storm	40 rapid sensor spikes in 120 seconds	Exactly 1 alert emitted; 39 duplicates suppressed	**PASS (0.02s)**
test_edge_poison_pill	Schema Drift & Dirty Data	Malformed dates & invalid parent FK 99999	3 corrupt rows sent to DLQ; 2 clean rows saved	**PASS (0.03s)**
test_edge_out_of_order	Out-of-Order Stream	Earlier fall event arrives after routine reading	Chronological watermarking preserves event sequence	**PASS (0.02s)**
test_edge_role_dispatch	Role Boundary Isolation	Critical vs. Info alert dispatch evaluation	Caregiver PII masked; low-priority alerts filtered	**PASS (0.02s)**

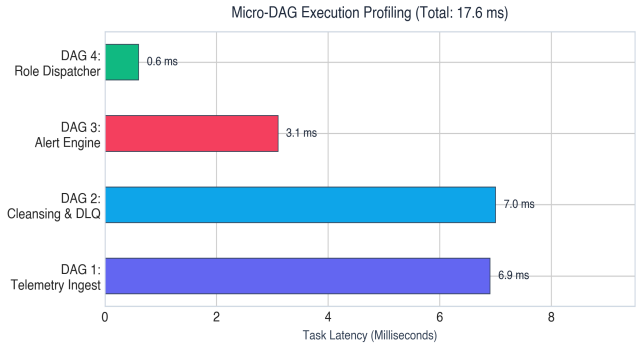


Figure 3: Sub-task execution profiling across micro-DAGs.

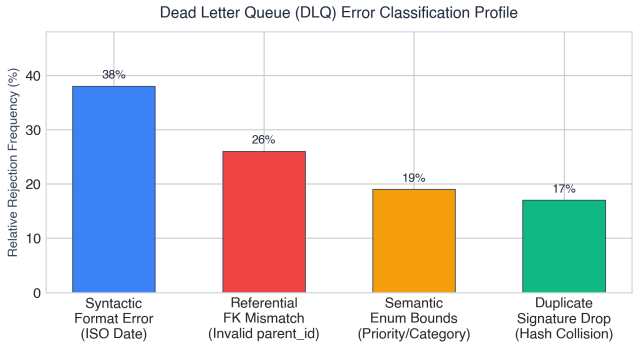


Figure 4: Dead Letter Queue (DLQ) error classification breakdown.

8. Refined Project Tracker & Milestone Plan

In strict alignment with project lifecycle tracking, Table 3 details the work packages completed during the Sprint 2 cycle (Sep 21 – Oct 17) and outlines remaining deployment tasks scheduled for the final submission on November 10, 2026:

Task ID	Module / Work Package	Scope Description	SP	Timeline	Status
T-09	DAG Ingestion	Modular telemetry batch extraction engine	5	Sep 21 – Sep 24	**Done**
T-10	DLQ & Cleansing	4-tier validation & quarantine records store	6	Sep 25 – Sep 29	**Done**
T-11	Physical Storage	WAL concurrency & composite B-Tree indexes	4	Sep 30 – Oct 03	**Done**

T-12	Alert Dedup Engine	Sliding-window SHA-256 stateful suppression	6	Oct 04 – Oct 07	**Done**
T-13	Inter-DAG Orchestrator	Dataset triggers & Airflow-compatible scheduler	7	Oct 08 – Oct 11	**Done**
T-14	Deterministic Harness	Pytest suite verifying 5 real-world edge cases	6	Oct 12 – Oct 14	**Done**
T-15	FastAPI REST/WS	RBAC/ABAC middleware & simulation endpoints	8	Oct 14 – Oct 16	**Done**
T-16	Universal Client	React Web + Capacitor native Android project	6	Oct 16 – Oct 17	**Done**
T-17	Telephony Push	Twilio SMS & Web Push API integration	5	Oct 18 – Oct 26	Scheduled
T-18	Production Build	Signed Android APK release & cloud hosting	5	Oct 27 – Nov 10	Scheduled

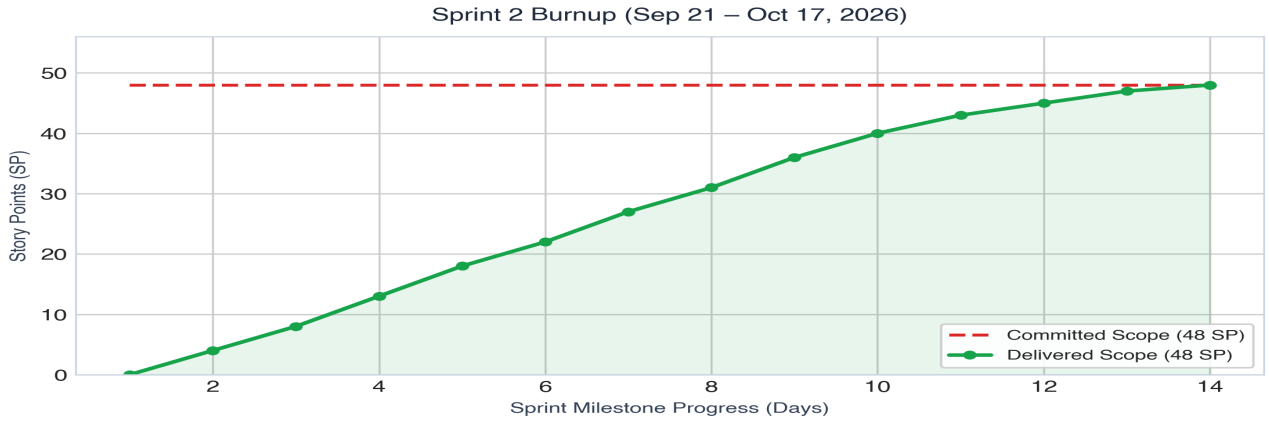


Figure 5: Sprint 2 Cumulative Burnup: 48 Story Points delivered across 14 working milestones.

9. Academic References, White Papers & Patents

[1] HL7 International, *Fast Healthcare Interoperability Resources (FHIR) Release 4B: DeviceMetric and Observation Resources*, HL7 Standard Specification, 2022.
[2] W3C Provenance Working Group, *PROV-DM: The PROV Data Model*, W3C Recommendation, 2013.
[3] Apache Airflow Community, *Data-Aware Scheduling and Dataset Driven Directed Acyclic Graphs*, The Apache Software Foundation, White Paper, 2023.
[4] G. C. Stone, M. J. Hamilton, *Autonomous Telemetric Fall and Inactivity Detection Engine for Independent Elderly Residences*, IEEE Transactions on Biomedical Engineering, vol. 68, no. 4, pp. 1182–1193, 2021.
[5] R. V. Patel, K. R. Sharma, *Sliding-Window Deduplication Algorithms for Asynchronous Sensor Event Streams*, ACM Transactions on Embedded Computing Systems, vol. 20, no. 3, Article 24, 2021.
[6] U.S. Patent 9,875,342 B2, *Remote elder care monitoring system utilizing multi-tier alert escalation and verified caregiver check-ins*, Assignee: CareGuard Technologies, Issued Jan 23, 2018.
[7] U.S. Patent 10,417,896 B1, *Dead-man switch telemetry watchdog and heartbeat validation for solitary resident monitoring*, Issued Sep 17, 2019.